

Lab7 Quad2SSA

1. quad2ssa

程序入口，对于函数列表的每个函数依次操作：`placePhi`，`renameVariables`，`cleanupUnusedPhi`。最后把每次处理后的函数再全部放入 `new vector<QuadFuncDecl*>` 中，然后打包为 `QuadProgram` 返回。

2. placePhi

放置 Phi 函数。

2.1 预处理

调用 `collectTempTypes` 遍历函数语句，记录每个 Temp 是 INT 还是 PTR。用编号到类型的 map 记录。

调用 `collectDefinedVars` 遍历函数语句，得到 `defsites[a]`，即每个变量在哪个块中被定义过。

2.2 寻找应该插入 Phi 函数的块

枚举每个变量 a ，将 `worklist` 初始化为其对应的 `defsites`。然后从 `worklist` 中取出一个块 n ，枚举所有这个块的 DF。这里我多加了一个剪枝，对于枚举的一个 DF，如果当前枚举的变量 a 不在这个块的 livein 中，那么说明，这个变量 a 在块 n 中的定义，并不会在这个 DF 之后被使用，也因此不需要在这里插入 Phi 函数。同时用 `map<int, set<int>> placedVarsByBlock` 记录一个块中已经插入了哪些变量的 Phi 函数，如果有则跳过，没有则插入 Phi 函数。

2.3 插入 Phi 函数

`QuadPhi` 包含一个最终变量 `QuadTemp`，一个 `vector<pair<Temp*, Label*>>` 表示每一个来自的前驱对应哪个 Temp，然后是 `def` 和 `use` 集合。

这里插入这个变量原始的 Temp 占位。前驱对应的 Temp 同样是由变量原始 Temp 占位。在下一步重命名中会给 Temp 加入版本来使其真正生效。

最后，如果这个块没有被加入过 `worklist`，将其加入 `worklist`，因为现在它也定义了变量块（通过 Phi 函数选择了对应的定义）。

3. renameVariables

变量重命名，添加版本。

3.0 结构

创建 `RenameContext`，维护重命名中 DFS 过程中所需的上下文信息。

1. `map<int, vector<Temp*>> versionStacks`：每个原始变量对应一个版本栈。栈顶就是当前作用域下这个变量可见的最新 SSA 版本。定义时压栈，退出块时把这个块压的都弹出。
2. `map<int, int> versionCounters`：记录每个变量对应的版本 count。
3. `visitedBlocks`：DFS 时标记经过。

3.1 初始化

把函数的参数作为变量，先把原版 Temp 压进版本栈中。Count 初始化为 -1。创建一个空的 `vector<int> pushedTemps` 来记录压栈的 Temp 的原编号，以便在 dfs 末尾将其 popback。

3.2 renameBlock

第一步，跳过块首的 Label 语句，然后处理块首的连续 Phi 语句。先获取原始变量 id（不含版本），然后调用 `pushNewVersion` 来完成获取新版本、压栈和增加 Count。用带版本的 Temp 替换原来 Phi 语句中的 temp_exp。

第二步，处理块内语句，先调用 `renameTerm` 修改语句右侧的 src 为当前版本值，再对左侧的 dst 调用 `pushNewVersion` 获取新版本值，以确保例如 `a=a+1` 的语句可以被正确设置版本。对于 Call、ExtCall，分别调用 `renameCallTerm` 和 `renameCallExtTerm` 来递归地修改对象指针和参数中的 temp。

第三步，处理完块内语句后，回填后继块里 Phi 函数的“来自本块的参数”。调用 `renameSuccessorPhiArgs` 完成。它会遍历当前块的后继，跳过块首 Label，然后扫描连续的 Phi 语句，把语句中 args 里 label 为当前块的对应 Temp 改为当前版本栈中对应版本。

第四步，递归处理 domTree 上的子节点。

最后将当前块中新增的版本出栈。

4. cleanupUnusedPhi

清理无用的 Phi。如果某个 Phi 定义出来的结果除了 Phi 函数自己之外，没有任何其他语句使用，那它就是无用的，可以删。

在 `recomputeAllDefUse` 中，对函数中所有语句调用 `recomputeDefUse`，重新得到每个语句带版本号的 Temp 的 def/use 集合。

扫描所有的函数，用 `map<int, set<QuadStm*>> useSites` 记录每个 Temp 被哪些语句使用过。

逐块遍历，如果某个 Phi 函数的目标 Temp 没有被其他语句使用过，将这个 Phi 函数删除。然后更新块内语句。

整个过程使用一个 While 循环进行重复直到遍历所有块后都没有发生删除，说明已经清理干净。